Лабораторная работа №2

Прогноз значения при помощи нейронной сети.

Построение однослойной нейронной сети

Перцептрон — неплохая отправная точка, но его возможности ограничены. Следующий шаг заключается в том, чтобы заставить набор нейронов действовать как одно целое и посмотреть, чего удастся достигнуть. Давайте создадим однослойную нейронную сеть, которая состоит из независимых нейронов, воздействующих на входные данные для получения выходного результата.

Создайте новый файл Python и импортируйте следующие файлы.

```
import numpy as np
import matplotlib.pyplot as plt
import neurolab as nl
```

Мы будем использовать входные данные из файла `data_simple_nn.txt`. Каждая строка этого файла содержит четыре числа. Первые два числа обра-

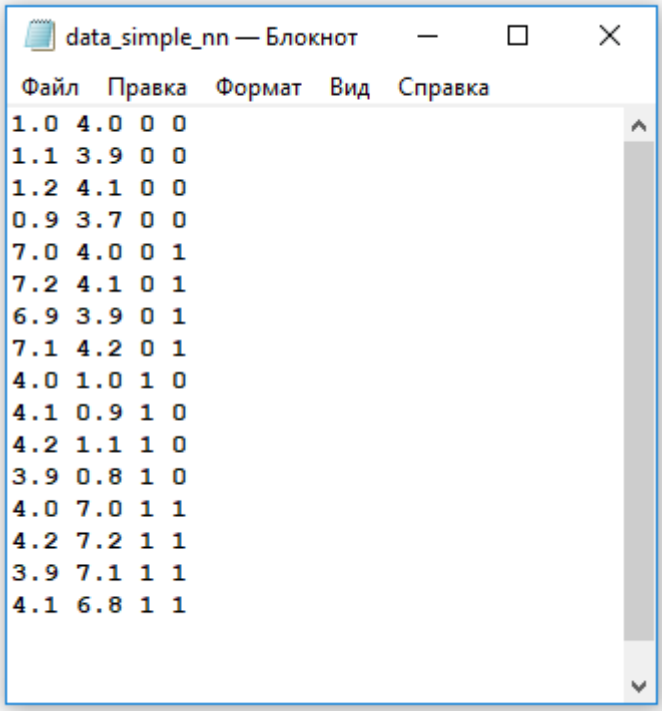

Рис. 14. Файл data_simple_nn.txt

зуют точку данных, а остальные два являются метками. Почему нам нужно использовать два числа в качестве меток? Потому что в нашем наборе данных имеются четыре различных класса, для представления которых нужны два бита. Пойдем дальше и загрузим данные.

```
# Загрузка входных данных
text = np.loadtxt('data_simple_nn.txt')
```

Разделим данные на точки данных и метки.

```
# Разделение данных на точки данных и метки
data = text[:, 0:2]
labels = text[:, 2:]
```

Построим график входных данных.

```
# Построение графика входных данных
plt.figure()
plt.scatter(data[:,0], data[:,1])
plt.xlabel('Размерность 1')
plt.ylabel('Размерность 2')
plt.title('Входные данные')
```

Извлечем минимальное и максимальное значения для каждого измерения (нам не нужно указывать их в коде, как это делалось в предыдущем разделе).

```
# Минимальное и максимальное значения для каждого измерения
dim1_min, dim1_max = data[:,0].min(), data[:,0].max()
dim2_min, dim2_max = data[:,1].min(), data[:,1].max()
```

Определим количество нейронов в выходном слое.

```
# Определение количества нейронов в выходном слое
num_output = labels.shape[1]
```

Определим однослойную нейронную сеть, используя заданные выше параметры.

```
# Определение однослойной нейронной сети
dim1 = [dim1_min, dim1_max]
dim2 = [dim2_min, dim2_max]
nn = nl.net.newp([dim1, dim2], num_output)
```

Обучим нейронную сеть с помощью тренировочных данных.

```
# Обучение нейронной сети
error_progress = nn.train(data, labels, epochs=100, show=20, lr=0.03)
```

Построим график продвижения процесса обучения.

```
# Построение графика продвижения процесса обучения
plt.figure()
plt.plot(error_progress)
plt.xlabel('Количество эпох')
plt.ylabel('Ошибка обучения')
plt.title('Изменение ошибки обучения')
plt.grid()

plt.show()
```

Определим выборочные тестовые точки данных и запустим для них нейронную сеть.

```
# Выполнение классификатора на тестовых точках данных
print('\nTest results:')
data_test = [[0.4, 4.3], [4.4, 0.6], [4.7, 8.1]]
for item in data_test:
    print(item, '-->', nn.sim([item])[0])
```

В процессе выполнения этого кода на экране отобразятся два графика. На первом снимке экрана представлены входные точки данных (рис. 14.3).

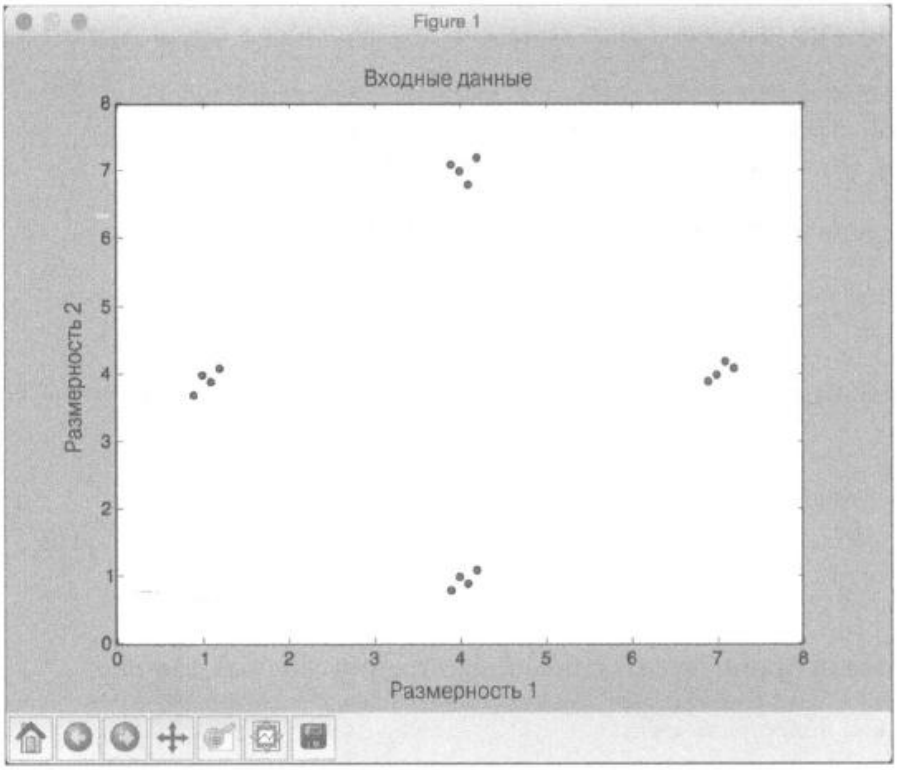

Рис. 14.3

На втором снимке экрана представлен график продвижения процесса обучения (рис. 14.4).

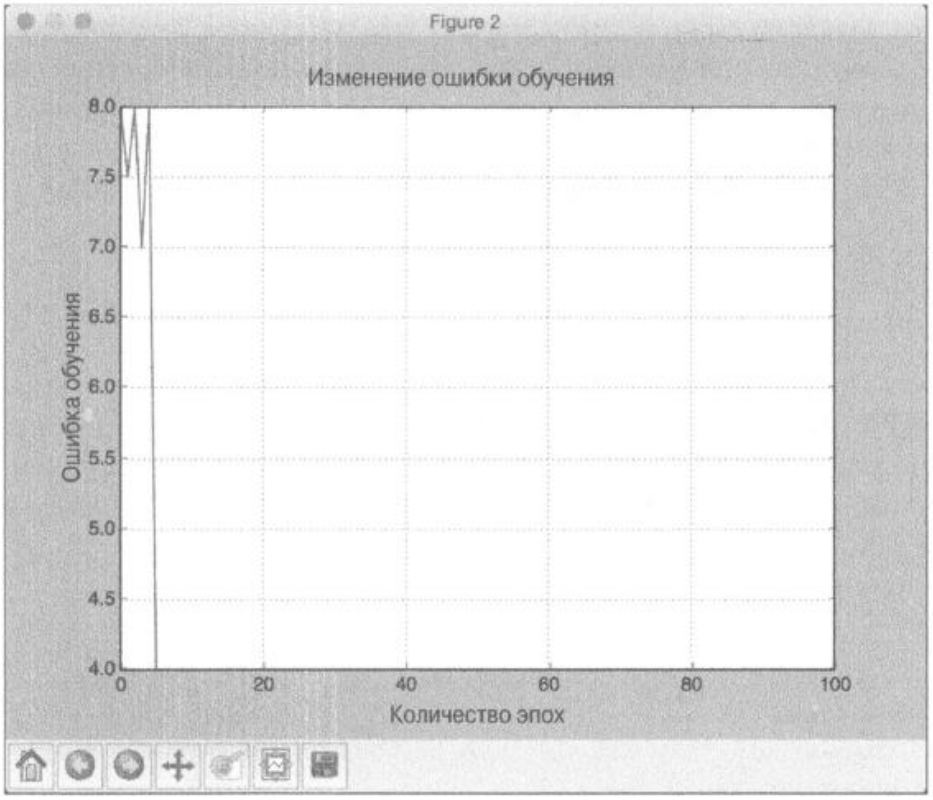

Рис. 14.4

Закрыв окна с этими графиками, вы увидите в окне терминала следующий вывод (рис. 14.5).

```
Epoch: 20; Error: 4.0;
Epoch: 40; Error: 4.0;
Epoch: 60; Error: 4.0;
Epoch: 80; Error: 4.0;
Epoch: 100; Error: 4.0;
The maximum number of train epochs is reached

Test results:
[0.4, 4.3] --> [ 0.  0.]
[4.4, 0.6] --> [ 1.  0.]
[4.7, 8.1] --> [ 1.  1.]
```

Рис. 14.5

Отыскав эти тестовые точки данных на двумерном графике, вы сможете убедиться в том, что результаты были предсказаны корректно.

Содержание работы.

Ответьте на вопросы и выполните следующие задания:

1. **Что называют однослойной нейронной сетью?**
2. **Что понимается под обучением нейронной сети?**
3. **Как происходит обучение однослойной нейронной сети на основе перцептрона?**
4. **Привести код на Python модели однослойной нейронной сети, рассмотренной в методических указаниях.**
5. **Самостоятельно провести эксперименты с однослойной нейронной сетью на иных произвольных входных данных (файл data_simple_nn.txt). Представить график продвижения процесса обучения.**

Приложение 1

```
import numpy as np
import matplotlib.pyplot as plt
import neurolab as nl

# Load input data
text = np.loadtxt('data_simple_nn.txt')

# Separate it into datapoints and labels
data = text[:, 0:2]
labels = text[:, 2:]

# Plot input data
plt.figure()
plt.scatter(data[:,0], data[:,1])
plt.xlabel('Dimension 1')
plt.ylabel('Dimension 2')
plt.title('Input data')

# Minimum and maximum values for each dimension
dim1_min, dim1_max = data[:,0].min(), data[:,0].max()
dim2_min, dim2_max = data[:,1].min(), data[:,1].max()

# Define the number of neurons in the output layer
num_output = labels.shape[1]

# Define a single-layer neural network
dim1 = [dim1_min, dim1_max]
dim2 = [dim2_min, dim2_max]
nn = nl.net.newp([dim1, dim2], num_output)

# Train the neural network
error_progress = nn.train(data, labels, epochs=100, show=20, lr=0.03)

# Plot the training progress
plt.figure()
plt.plot(error_progress)
plt.xlabel('Number of epochs')
plt.ylabel('Training error')
plt.title('Training error progress')
plt.grid()

plt.show()

# Run the classifier on test datapoints
print('\nTest results:')
data_test = [[0.4, 4.3], [4.4, 0.6], [4.7, 8.1]]
for item in data_test:
    print(item, '-->', nn.sim([item])[0])
```

Рис. 14.П1. Пример кода модели однослойной нейронной сети на Python

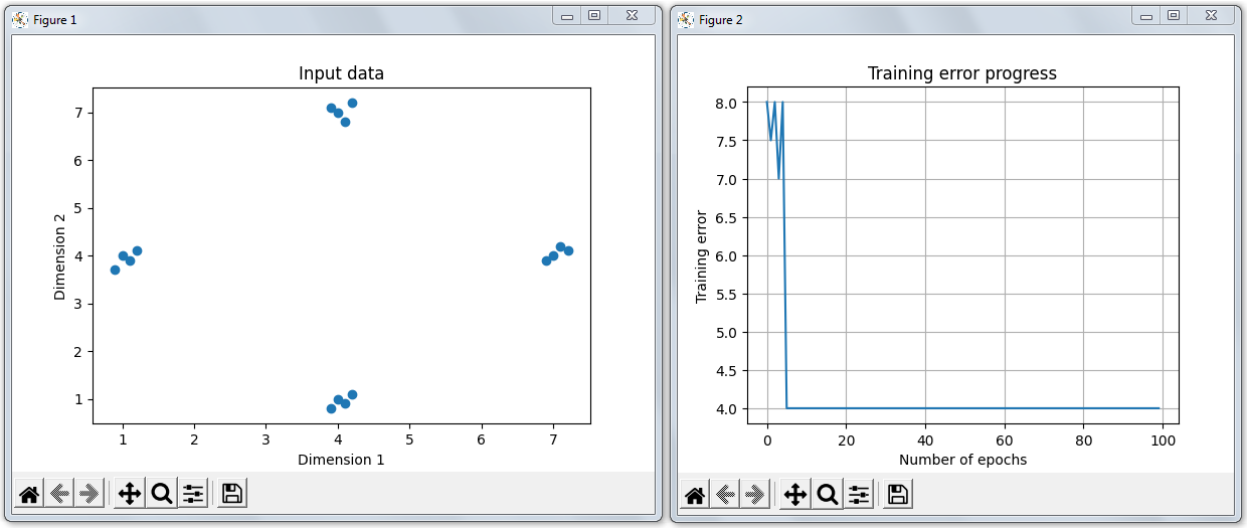

Рис. 14.П2. Входные точки данных и изменение ошибки обучения в зависимости от числа наблюдаемых эпох

```
Epoch: 20; Error: 4.0;
Epoch: 40; Error: 4.0;
Epoch: 60; Error: 4.0;
Epoch: 80; Error: 4.0;
Epoch: 100; Error: 4.0;
The maximum number of train epochs is reached

Test results:
[0.4, 4.3] --> [0. 0.]
[4.4, 0.6] --> [1. 0.]
[4.7, 8.1] --> [1. 1.]
```

Рис. 14.П3. Результат выполнения кода